Table 6.7 A comparison of authorization models

Model	What Drives Access Decisions	How Policies Are Expressed	Key Strengths
ACL	Lists attached to each resource, mapping principals to actions	Data (the list itself)	Simple and intuitive; fast at runtime
RBAC	Principals are assigned to roles, roles map to permissions	Data (roles and role assignments)	Aligns with org structure; easy to audit
ReBAC	Access depends on paths through a graph of relationships	Relationship data + rules	Matches human concepts of ownership, delegation
ABAC	Policies use attributes of principal, resource, action, and environment	Policy logic evaluated at runtime using contextual attributes	Fine-grained, contextual, scalable

The models we've discussed are not mutually exclusive. Like ACME, most modern organizations use a hybrid approach—that we'll call Policy-Based Access Control (PBAC)—applying RBAC for broad job responsibilities, ABAC for detailed control based on context, and ReBAC for collaboration platforms or delegated access. Understanding each model's design and tradeoffs provides teams with the vocabulary and framework they need to evaluate their systems and create access decisions that reflect real-world complexity.

In the next chapter, we focus on the fundamental components of policies. You've seen policies expressed through lists, graphs, and code. Now, we'll examine more closely how modern systems organize policies, what they assess, how they define scope and effects, and how they handle authorization logic within a larger system. We'll introduce Cedar, an open-source policy language developed by AWS, as a running example to demonstrate core policy concepts and patterns.

6.4 Summary

- ACLs, RBAC, ReBAC, and ABAC each offer distinct approaches to authorization. They differ in how access decisions are described and enforced, ranging from simple lists to dynamic, context-aware policies.
- ReBAC bases access on relationships between entities, such as ownership or team membership. This model works especially well in